

Programming Fundamentals

Short notes in C and Python · Equigrow learning path, step 1

Read one section, then write the code yourself. Typing it out teaches far more than reading it.

1. Your first program

C

```
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

Python

```
print("Hello, world!")
```

- **C** is compiled: `gcc hello.c -o hello`, then run `./hello`. Every program starts in `main()`.
- **Python** is interpreted: run `python hello.py`. No main function or semicolons needed.
- C statements end with `;` and blocks use `{ }`. Python uses indentation (4 spaces) for blocks.

2. Variables & data types

A variable is a named box that stores a value. C needs the type written out; Python works it out for you.

Kind	C	Python	Example
Whole number	int	int	42
Decimal	float / double	float	3.14
Single character	char	str (length 1)	'A'
Text	char[] (array)	str	"Equigrow"
True / false	int (0/1) or bool*	bool	True

* bool needs `#include <stdbool.h>` in C.

C

```
int age = 18;
float cgpa = 8.5;
char grade = 'A';
printf("%d %.1f %c\n", age, cgpa, grade);
```

Python

```
age = 18
cgpa = 8.5
grade = "A"
print(age, cgpa, grade)
```

Input: C uses `scanf("%d", &age);` (note the `&`). Python uses `age = int(input())`, because `input()` always returns text.

3. Operators

Type	Operators	Watch out
Arithmetic	+ - * / %	In C, <code>7/2</code> is 3 (integer division). In Python, <code>7/2</code> is 3.5 and <code>7//2</code> is 3.
Comparison	<code>== != < > <= >=</code>	<code>=</code> assigns a value; <code>==</code> compares two values.
Logical	C: <code>&& !</code> Python: <code>and or not</code>	Stops early: in a <code>&& b</code> , <code>b</code> is skipped if <code>a</code> is false.
Assignment	<code>= += -= *=</code>	C also has <code>++</code> and <code>--</code> . Python does not.

4. Conditions

C

```
if (marks >= 90) {
    printf("A\n");
} else if (marks >= 75) {
    printf("B\n");
} else {
    printf("Keep going\n");
}
```

Python

```
if marks >= 90:
    print("A")
elif marks >= 75:
    print("B")
else:
    print("Keep going")
```

5. Loops

Use **for** when you know how many times to repeat, and **while** when you repeat until something changes.

C

```
// print 1 to 5
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}

int n = 10;
while (n > 0) {
    n = n / 2;
}
```

Python

```
# print 1 to 5
for i in range(1, 6):
    print(i, end=" ")

n = 10
while n > 0:
    n = n // 2
```

- **break** leaves the loop immediately; **continue** skips to the next round.
- `range(1, 6)` gives 1 to 5: the end value is **not** included.

6. Functions

A function is a reusable block of code. Give it inputs (parameters), and it can send back a result with **return**.

C

```
int square(int x) {
    return x * x;
}

int main(void) {
    printf("%d\n", square(4)); // 16
    return 0;
}
```

Python

```
def square(x):
    return x * x

print(square(4)) # 16
```

Tip: one function should do one job. If you're copying and pasting code, turn it into a function.

7. Arrays, lists & strings

C

```
int marks[5] = {90, 78, 85, 60, 95};
int sum = 0;
for (int i = 0; i < 5; i++) {
    sum += marks[i];
}
printf("Average: %.2f\n", sum / 5.0);

char name[] = "Asha"; // ends with '\0'
// strlen(name) is 4 (needs <string.h>)
```

Python

```
marks = [90, 78, 85, 60, 95]
print("Average:", sum(marks) / len(marks))

marks.append(88) # lists can grow
print(marks[0], marks[-1]) # first, last

name = "Asha"
print(len(name), name.upper())
```

- Indexes start at **0**. The last element of a size-5 array is at index 4.
- C arrays have a fixed size and no bounds checking: reading marks[5] is a bug. Python lists grow and raise an error if you go out of range.
- C strings are character arrays ending in '\0'. Python strings can't be changed in place; methods return new strings.

8. Pointers & memory (C)

A pointer stores the **address** of another variable. Python handles memory for you, but C lets you work with it directly.

```
int x = 10;
int *p = &x; // p holds the address of x
printf("%d\n", *p); // *p reads the value at that address: 10
*p = 20; // changes x through the pointer
printf("%d\n", x); // 20

void swap(int *a, int *b) { // pass addresses to change the caller's variables
    int t = *a; *a = *b; *b = t;
}
swap(&x, &y);

int *arr = malloc(5 * sizeof(int)); // memory at run time (needs <stdlib.h>)
if (arr == NULL) { /* handle the error */ }
free(arr); // always free what you malloc
```

Symbol	Meaning
&x	address of x
*p	value stored at the address in p
int *p	p is a pointer to an int
NULL	a pointer that points nowhere

9. File handling

C

```
FILE *f = fopen("notes.txt", "w");
if (f != NULL) {
    fprintf(f, "Hello file\n");
    fclose(f);
}
```

Python

```
with open("notes.txt", "w") as f:
    f.write("Hello file\n")

with open("notes.txt") as f:
    for line in f:
        print(line.strip())
```

Modes: "r" read, "w" write (replaces the file), "a" append. Python's with closes the file for you.

10. Common beginner mistakes

- Using `=` instead of `==` inside an `if`.
- Off-by-one errors: looping to `i <= n` on an array of size `n`.
- Forgetting `&` in `scanf`, or forgetting to convert `input()` to `int` in Python.
- Wrong indentation in Python: it changes which lines belong to the loop.
- Using a variable before giving it a value (C won't always warn you).

11. Practice (try in both languages)

- Print whether a number is even or odd.
- Find the largest of three numbers.
- Print the multiplication table of `n`.
- Reverse a string without a built-in function.
- Count vowels in a sentence.
- Find the sum and average of `n` marks stored in an array.
- Check if a number is prime.
- Write a function that returns the factorial of `n`.
- Swap two numbers using pointers (C).
- Read a file and count how many lines it has.

Practise on HackerRank (C and Python tracks) and watch the Neso Academy C lessons linked in Equigrow. Mark this step as learned when you can solve these without help.